# 宠物领养模块讲解文档

## 一、数据库与实体类设计

### 1.1 核心实体表设计

#### 1.1.1 领养申请表(adoption)

| 字段名 | 数据类型 | 描述 | 约束条件 |
| --- | --- | --- | --- |
| id | BIGINT | 申请ID | 主键，自增 |
| user_id | BIGINT | 用户ID | 外键关联user表 |
| pet_id | BIGINT | 宠物ID | 外键关联pet表 |
| status | VARCHAR(20) | 状态(已申请/审核中/已通过/已拒绝) | 非空 |
| apply_reason | TEXT | 申请理由 | 可选 |
| contact_phone | VARCHAR(20) | 联系电话 | 非空 |
| address | VARCHAR(255) | 地址 | 非空 |
| create_time | DATETIME | 创建时间 | 自动生成 |
| update_time | DATETIME | 更新时间 | 自动更新 |

**Adoption类：对应领养申请表，包含领养申请基本信息**
【springboot/src/main/java/org/example/springboot/entity/Adoption.java】

```
@Data
@TableName("adoption")
@Schema(description = "领养申请")
public class Adoption {

    @TableId(type = IdType.AUTO)
    @Schema(description = "申请ID")
    private Long id;
```

半夜撕代码版权所有

```
    @Schema(description = "用户ID")
    private Long userId;

    @Schema(description = "宠物ID")
    private Long petId;

    @Schema(description = "状态(已申请/审核中/已通过/已拒绝)")
    private String status;

    @Schema(description = "申请理由")
    private String applyReason;

    // 其他属性...
}
```

### 1.1.2 宠物表(pet)

| 字段名 | 数据类型 | 描述 | 约束条件 |
|---|---|---|---|
| id | BIGINT | 宠物ID | 主键，自增 |
| name | VARCHAR(50) | 宠物名称 | 非空 |
| category_id | BIGINT | 宠物分类ID | 外键关联category表 |
| breed | VARCHAR(50) | 品种 | 可选 |
| age | INTEGER | 年龄 | 可选 |
| gender | VARCHAR(10) | 性别 | 可选 |
| health_status | VARCHAR(50) | 健康状况 | 可选 |
| description | TEXT | 描述 | 可选 |
| images | VARCHAR(1000) | 图片(多张图片用逗号分隔) | 可选 |
| adoption_status | VARCHAR(20) | 领养状态(可领养/已领养) | 非空 |
| create_time | DATETIME | 创建时间 | 自动生成 |
| update_time | DATETIME | 更新时间 | 自动更新 |

半夜撕代码版权所有

**Pet类**：对应宠物表，包含宠物基本信息
【springboot/src/main/java/org/example/springboot/entity/Pet.java】

```java
@Data
@TableName("pet")
@Schema(description = "宠物信息")
public class Pet {

    @TableId(type = IdType.AUTO)
    @Schema(description = "宠物ID")
    private Long id;

    @Schema(description = "宠物名称")
    private String name;

    @Schema(description = "宠物分类ID")
    private Long categoryId;

    @Schema(description = "分类名称")
    @TableField(exist = false)
    private String categoryName;

    // 其他属性...
}
```

### 1.2 核心属性说明

1. **宠物领养状态**：宠物有两种领养状态，分别是"可领养"和"已领养"，通过AdoptionStatus枚举类定义
2. **申请状态流转**：领养申请有四种状态，分别是"已申请"、"审核中"、"已通过"和"已拒绝"，通过ApplicationStatus枚举类定义
3. **宠物图片**：支持多图存储，images字段中存储图片URL，多个图片之间使用逗号分隔
4. **关联关系**：通过user_id和pet_id字段建立与用户表和宠物表的关联关系

# 二、各功能详细讲解

## 2.1 宠物领养申请功能

### 2.1.1 功能概述

宠物领养申请功能允许用户在浏览宠物详情时，对状态为"可领养"的宠物提交领养申请。用户需要填写申请理由、联系电话和地址等信息，系统会验证用户登录状态和宠物可领养状态，确保申请的合法性。

半夜撕代码版权所有

### 2.1.2 详细讲解功能实现流程

用户在宠物详情页面查看宠物信息，如果宠物状态为"可领养"，页面会显示"申请领养"按钮。用户点击按钮后，系统首先检查用户是否已登录，未登录则引导用户前往登录页面。登录后，系统弹出领养申请表单，用户填写申请理由、联系电话和地址等信息。

用户提交表单后，前端通过request.post('/adoption')将数据发送到后端。后端AdoptionController接收请求，从JWT令牌中获取当前用户ID，然后调用adoptionService.createAdoption方法处理申请。该方法首先验证用户和宠物的存在性，检查宠物是否处于可领养状态，并确保用户没有重复申请同一只宠物。验证通过后，创建新的领养申请记录，设置初始状态为"已申请"，并保存到数据库。

申请成功后，前端会显示成功提示，并刷新宠物详情页面，更新宠物的申请状态。用户可以在"我的领养"页面查看申请进度。

### 2.1.3 关键代码讲解

1. 前端领养申请表单
   【vue3/src/views/frontend/pet/PetDetail.vue】

```
// 处理申请领养
const handleAdopt = () => {
  if (!userStore.isLoggedIn) {
    ElMessage.warning(t('pet.loginRequired'))
    router.push({ path: '/login', query: { redirect: route.fullPath }
})
    return
  }

  dialogVisible.value = true
}

// 提交领养申请
const submitAdoption = () => {
  adoptFormRef.value.validate(async (valid) => {
    if (!valid) return

    submitLoading.value = true
    try {
      await request.post('/adoption', adoptForm.value, {
        successMsg: t('pet.applicationSubmitted'),
        onSuccess: () => {
          dialogVisible.value = false
          fetchPetDetail() // 刷新详情
        }
      })
    } catch (error) {
      console.error(t('pet.errors.submitApplicationFailed'), error)
    } finally {
```

```
        submitLoading.value = false
      }
    })
  }
```

这段代码实现了领养申请的前端逻辑。handleAdopt方法首先检查用户登录状态，未登录则跳转到登录页面；submitAdoption方法验证表单数据，然后将申请信息提交到后端，成功后关闭对话框并刷新宠物详情。

2. **后端领养申请处理**
   【springboot/src/main/java/org/example/springboot/service/AdoptionService.java】

```
@Transactional
public void createAdoption(Long userId, AdoptionDTO adoptionDTO) {
    // 验证用户是否存在
    User user = userMapper.selectById(userId);
    if (user == null) {
        throw new ServiceException("用户不存在");
    }

    // 验证宠物是否存在
    Pet pet = petMapper.selectById(adoptionDTO.getPetId());
    if (pet == null) {
        throw new ServiceException("宠物不存在");
    }

    // 验证宠物是否可领养
    if
(!AdoptionStatus.AVAILABLE.getValue().equals(pet.getAdoptionStatus()))
{
        throw new ServiceException("该宠物当前不可领养");
    }

    // 检查用户是否已经申请过该宠物
    LambdaQueryWrapper<Adoption> queryWrapper = new
LambdaQueryWrapper<>();
    queryWrapper.eq(Adoption::getUserId, userId);
    queryWrapper.eq(Adoption::getPetId, adoptionDTO.getPetId());
    Adoption existAdoption = adoptionMapper.selectOne(queryWrapper);

    if (existAdoption != null) {
        throw new ServiceException("您已经申请过该宠物，请勿重复申请");
    }

    // 创建申请
    Adoption adoption = new Adoption();
    adoption.setUserId(userId);
    adoption.setPetId(adoptionDTO.getPetId());
```

```
        adoption.setStatus(ApplicationStatus.APPLIED.getValue());
        adoption.setApplyReason(adoptionDTO.getApplyReason());
        adoption.setContactPhone(adoptionDTO.getContactPhone());
        adoption.setAddress(adoptionDTO.getAddress());
        adoption.setCreateTime(LocalDateTime.now());
        adoption.setUpdateTime(LocalDateTime.now());

        adoptionMapper.insert(adoption);
    }
```

这段代码实现了领养申请的后端处理逻辑。首先验证用户和宠物的存在性，检查宠物是否可领养，确保用户没有重复申请，然后创建新的领养申请记录并设置初始状态为"已申请"，最后保存到数据库。

## 2.2 我的领养申请功能

### 2.2.1 功能概述

我的领养申请功能允许用户查看自己的所有领养申请及其状态。用户可以按状态筛选申请记录，查看详情，对"已申请"或"审核中"状态的申请进行取消操作，并查看相关宠物信息。该功能提供了领养申请全生命周期的可视化管理。

### 2.2.2 详细讲解功能实现流程

用户进入"我的领养"页面时，MyAdoption.vue组件初始化并调用fetchAdoptions方法。该方法首先检查用户登录状态，未登录则提示用户登录；已登录则通过request.get('/adoption/user')向后端发送请求，传递可选的状态筛选条件和分页参数。

后端AdoptionController接收请求，从JWT令牌中获取当前用户ID，然后调用adoptionService.getUserAdoptions方法查询该用户的领养申请记录。查询结果按创建时间倒序排列，并进行分页处理，然后返回给前端。

前端接收数据后，将领养申请记录渲染为时间线形式，显示申请ID、状态、理由、联系方式和地址等信息。用户可以通过顶部的单选按钮组筛选不同状态的申请。对于"已申请"或"审核中"状态的申请，用户可以点击"取消"按钮进行取消操作，取消操作会调用request.delete(`/adoption/${id}`)发送请求，后端验证用户身份并删除申请记录。

### 2.2.3 关键代码讲解

1. **前端领养申请列表获取**
   【vue3/src/views/frontend/pet/MyAdoption.vue】

```
// 获取领养申请列表
const fetchAdoptions = async () => {
  // 确保初始化为空数组而不是undefined
  adoptionList.value = []

  if (!userStore.isLoggedIn) {
```

```
    ElMessageBox.confirm(t('adoption.loginRequired'), t('common.tip'),
{
      confirmButtonText: t('adoption.goToLogin'),
      cancelButtonText: t('common.cancel'),
      type: 'warning'
    }).then(() => {
      router.push({
        path: '/login',
        query: { redirect: router.currentRoute.value.fullPath }
      });
    }).catch(() => {
      router.push('/');
    });
    return;
  }

  loading.value = true
  loadError.value = false
  try {
    await request.get('/adoption/user', {
      status: currentStatus.value,
      currentPage: currentPage.value,
      size: pageSize.value
    }, {
      onSuccess: (res) => {
        adoptionList.value = res.records
        total.value = res.total
      },
      onError: () => {
        loadError.value = true
      }
    })
  } catch (error) {
    console.error('获取领养申请列表失败:', error)
    loadError.value = true
  } finally {
    loading.value = false
  }
}
```

这段代码实现了获取用户领养申请记录的前端逻辑。首先检查用户登录状态，未登录则提示用户登录；已登录则发送请求获取数据，包含状态筛选和分页参数，成功后更新申请列表和总数。

2. **后端用户领养申请查询**
   【springboot/src/main/java/org/example/springboot/service/AdoptionService.java】

半夜撕代码版权所有

```java
public Page<Adoption> getUserAdoptions(Long userId, String status,
Integer currentPage, Integer size) {
    LambdaQueryWrapper<Adoption> queryWrapper = new
LambdaQueryWrapper<>();
    queryWrapper.eq(Adoption::getUserId, userId);

    if (StringUtils.isNotBlank(status)) {
        queryWrapper.eq(Adoption::getStatus, status);
    }

    queryWrapper.orderByDesc(Adoption::getCreateTime);

    return adoptionMapper.selectPage(new Page<>(currentPage, size),
queryWrapper);
}
```

这段代码实现了根据用户ID查询领养申请记录的后端逻辑。构建查询条件，包括用户ID和可选的状态筛选，按创建时间倒序排序，最后执行分页查询并返回结果。

## 2.3 领养申请管理功能

### 2.3.1 功能概述

领养申请管理功能是后台管理系统的一部分，允许管理员查看和处理所有用户的领养申请。管理员可以按宠物ID、用户ID和状态筛选申请记录，查看详情，并对申请进行审核操作，包括通过或拒绝。该功能实现了领养申请的全流程管理。

### 2.3.2 详细讲解功能实现流程

管理员进入领养申请管理页面时，AdoptionManage.vue组件初始化并调用fetchAdoptions方法。该方法通过request.get('/adoption/page')向后端发送请求，传递筛选条件和分页参数。

后端AdoptionController接收请求，调用adoptionService.getAdoptionsByPage方法查询领养申请记录。查询结果按创建时间倒序排列，并进行分页处理，然后返回给前端。

前端接收数据后，将领养申请记录渲染为表格形式，显示ID、用户ID、宠物ID、申请理由、联系方式、地址和状态等信息。管理员可以对"已申请"或"审核中"状态的申请进行审核操作：

- 通过申请：点击"通过"按钮，调用request.put(`/adoption/${id}/review`)发送请求，传递status="已通过"参数
- 拒绝申请：点击"拒绝"按钮，调用request.put(`/adoption/${id}/review`)发送请求，传递status="已拒绝"参数

后端接收审核请求后，调用adoptionService.reviewAdoption方法处理。该方法更新申请状态，如果申请被通过，还会将宠物状态更新为"已领养"，并自动拒绝该宠物的其他申请。

### 2.3.3 关键代码讲解

1. 前端领养申请审核
   【vue3/src/views/backend/pet/AdoptionManage.vue】

```
// 通过申请
const handleApprove = (row) => {
  ElMessageBox.confirm(t('backend.adoption.messages.approveConfirm'),
t('common.tip'), {
    confirmButtonText: t('common.confirm'),
    cancelButtonText: t('common.cancel'),
    type: 'warning'
  }).then(async () => {
    try {
      await request.put(`/adoption/${row.id}/review`, null, {
        params: {
          status: '已通过'
        },
        successMsg: t('backend.adoption.messages.approveSuccess'),
        onSuccess: () => {
          fetchAdoptions()
        }
      })
    } catch (error) {
      console.error('审核申请失败:', error)
    }
  }).catch(() => {})
}
```

这段代码实现了领养申请审核的前端逻辑。通过点击"通过"按钮，弹出确认对话框，然后调用接口更新申请状态为"已通过"，成功后刷新列表。

2. 后端领养申请审核
   【springboot/src/main/java/org/example/springboot/service/AdoptionService.java】

```
@Transactional
public void reviewAdoption(Long id, String status) {
    Adoption adoption = getAdoptionById(id);

    // 更新申请状态
    adoption.setStatus(status);
    adoption.setUpdateTime(LocalDateTime.now());
    adoptionMapper.updateById(adoption);

    Pet pet = petMapper.selectById(adoption.getPetId());
    if (pet == null) {
        throw new ServiceException("宠物不存在");
    }
```

```
    // 根据审核结果更新宠物状态
    if (ApplicationStatus.APPROVED.getValue().equals(status)) {
        pet.setAdoptionStatus(AdoptionStatus.ADOPTED.getValue());
        petMapper.updateById(pet);

        // 拒绝其他申请
        rejectOtherApplications(pet.getId(), adoption.getId());
    }
    // 如果申请被拒绝，宠物状态保持为"可领养"
}

/**
 * 拒绝其他申请
 */
private void rejectOtherApplications(Long petId, Long
approvedAdoptionId) {
    LambdaQueryWrapper<Adoption> queryWrapper = new
LambdaQueryWrapper<>();
    queryWrapper.eq(Adoption::getPetId, petId);
    queryWrapper.ne(Adoption::getId, approvedAdoptionId);
    queryWrapper.ne(Adoption::getStatus,
ApplicationStatus.REJECTED.getValue());

    Adoption updateAdoption = new Adoption();
    updateAdoption.setStatus(ApplicationStatus.REJECTED.getValue());
    updateAdoption.setUpdateTime(LocalDateTime.now());

    adoptionMapper.update(updateAdoption, queryWrapper);
}
```

这段代码实现了领养申请审核的后端逻辑。首先更新申请状态，如果申请被通过，还会将宠物状态更新为"已领养"，并通过rejectOtherApplications方法自动拒绝该宠物的其他申请，确保一只宠物只能被一个用户领养。

# 三、总结

## 3.1 宠物领养申请功能实现

宠物领养申请功能通过前端PetDetail.vue组件和后端AdoptionController实现。用户在宠物详情页面点击"申请领养"按钮，填写申请表单后提交。后端验证用户身份、宠物可领养状态和申请唯一性，创建新的领养申请记录，设置初始状态为"已申请"。整个流程设计合理，确保了申请数据的有效性和一致性，为用户提供了便捷的领养申请体验。

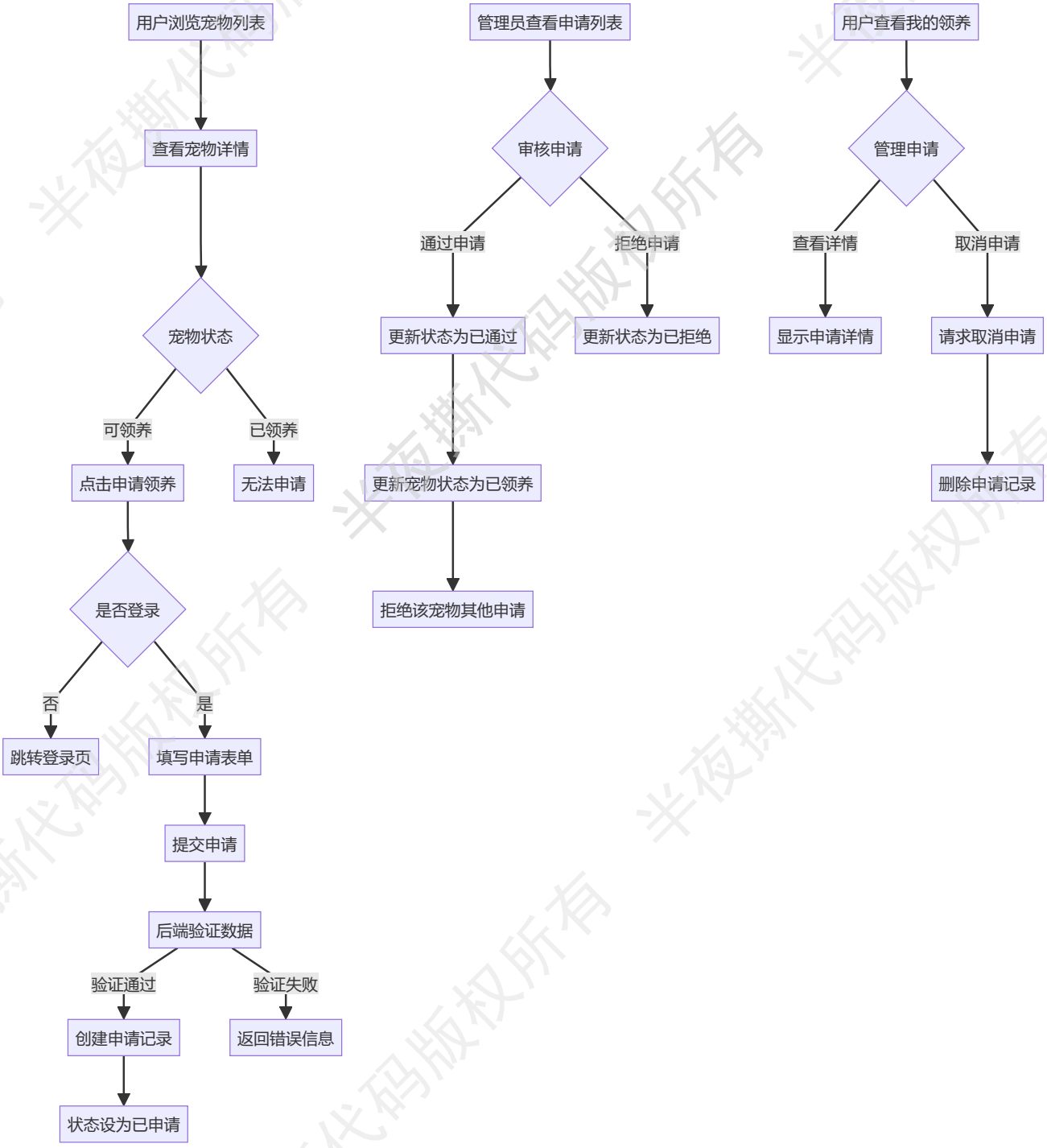
半夜撕代码版权所有

## 3.2 我的领养申请功能实现

我的领养申请功能通过MyAdoption.vue组件展示用户的领养申请历史。用户可以按状态筛选申请记录，查看详情，并对"已申请"或"审核中"状态的申请进行取消操作。后端通过userId条件查询，确保用户只能查看和操作自己的申请。界面采用时间线形式展示申请历史，状态使用不同颜色的标签区分，方便用户快速了解申请进展。

## 3.3 领养申请管理功能实现

领养申请管理功能通过AdoptionManage.vue组件为管理员提供全面的领养申请管理能力。管理员可以查看所有用户的领养申请，进行筛选和分页，并对申请进行审核操作。当管理员通过某个申请时，系统自动将宠物状态更新为"已领养"，并拒绝该宠物的其他申请，确保领养流程的完整性和数据一致性。整个管理界面设计简洁明了，提高了管理效率。




半夜撕代码版权所有